

2.39. Réduire un dessin : ÷2

Le bouton «  ÷2 » fait d'un dessin de 16 une copie de 8 : chaque carré de 2×2 devient un pixel. Des détails se perdent.

1. Ce qu'il faut comprendre

Le bouton  ÷2 fait l'inverse de $\times 2$: une copie **deux fois plus petite**. Un dessin de 32 devient un dessin de 16, un dessin de 16 devient un dessin de 8.

Comment il réduit : le dessin est découpé en **carrés de 2×2 pixels**, et chaque carré devient **un seul pixel**. Lequel des quatre garder ? Le **plus foncé** : ainsi un contour d'un pixel d'épaisseur ne disparaît pas.

Quatre pixels n'en font plus qu'un : des détails se perdent forcément. À gauche, `ROND_PETIT`, le rond du 2.01 réduit ; à droite, `ROND_8`, le rond redessiné en 8×8 (le 2.35). Comparez-les : le réduit est plus épais, plus foncé.

Pourquoi la vraie Game Boy n'a pas de zoom : elle n'a pas le temps de calculer de nouveaux pixels pendant qu'elle dessine l'écran, 60 fois par seconde. Les jeux de l'époque dessinaient donc chaque taille à la main — exactement ce que fait la galerie avec ses trois tailles.

Essaie : ouvre `tuile_ROND_PETIT.cpp` et éclaircis-le en changeant quelques `#` en `-`, pour qu'il ressemble au rond de droite.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <texte>` → [0.0.2. La fonction `texte\(\)` — écrire un mot](#)
- `#include <sprite>` → [0.69.4. La fonction `sprite\(\)` — un lutin au pixel près](#)
- `#include <Tuile>` → [0.107.2. Tuile — dessiner une case](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **501 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
démarrage          57 o ← la console, à l'allumage — la mise en route de la console
main()             78 o ← démarrage — ta fonction
VBlank              8 o ← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage       28 o ← image() — attend l'image suivante
AttendreVBlank      16 o ← démarrage — attend que l'écran soit libre
AttendreAcces       12 o ← EcrireTexte — attend de pouvoir écrire une case
EcrireTexte         10 o ← texte() — écrit un texte, lettre par lettre
RangerLutins        11 o ← démarrage — range les 40 lutins hors de l'écran
InstallerTransfert  15 o ← démarrage — installe la copie des lutins en mémoire rapide
CopierTuiles        67 o ← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte        14 o ← démarrage — vide les deux cartes de l'écran
Tuiles             160 o ← CopierTuiles — 8 lettres sur 43, et 2 tuiles dessinées
DonneesTransfert    10 o ← InstallerTransfert — la petite routine qui copie les lutins
"REDUIT"            6 o ← texte() — un texte gravé
"REDESSINE"         9 o ← texte() — un texte gravé
```

Lettres gravées : DEINRSTU

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, Hasard, EffacerFond, EffacerPanneau, LireManette, Notes

Fonctions du langage appelées :

```
sprite() × 2    46 o sur place, dans main()
texte() × 2     22 o sur place, dans main() + EcrireTexte (10 o)
image() × 1     6 o sur place, dans main() + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- 2.39 Réduire un dessin : ÷2 ----
// À gauche le rond réduit (chaque carré de 2 × 2 devient un pixel), à droite le rond redessiné.

#include <texte>           // texte() : écrire un mot
#include <sprite>          // sprite() : un dessin de 8 × 8
#include <Tuile>           // Tuile : 8 × 8
#include "tuile_ROND_PETIT.cpp" // le rond du 2.01, réduit par ÷2
#include "tuile_ROND_8.cpp"   // le rond redessiné en 8 × 8
```

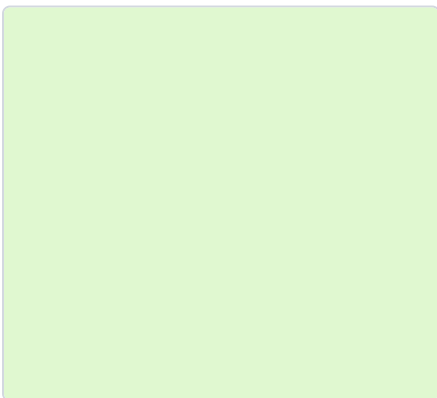
2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    sprite(0, 36, 60, ROND_PETIT); // le lutin 0
    sprite(1, 108, 60, ROND_8);    // le lutin 1
    texte(2, 10, "REDUIT");         // sous celui de gauche
    texte(10, 10, "REDESSINE");    // sous celui de droite

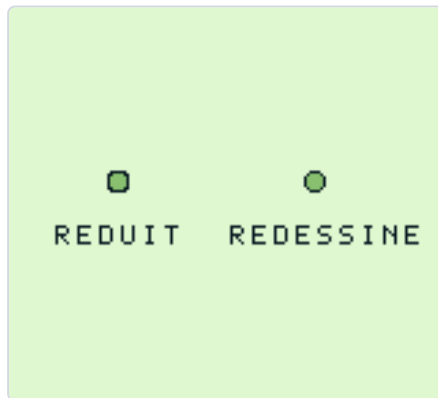
    while (true) {
        image();
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Deux petits ronds : à gauche plus foncé (REDUIT), à droite plus fin (REDESSINE).

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le rond réduit en (36, 60)

✓ le rond redessiné en (108, 60)

✓ les deux mots sont écrits

Ensuite : **2.40. La fonction `spriteTaille()` — dessiner une fois, choisir la taille** — série 2 : Les formes géométriques. · gameboy3
— écrire une cartouche Game Boy en C++